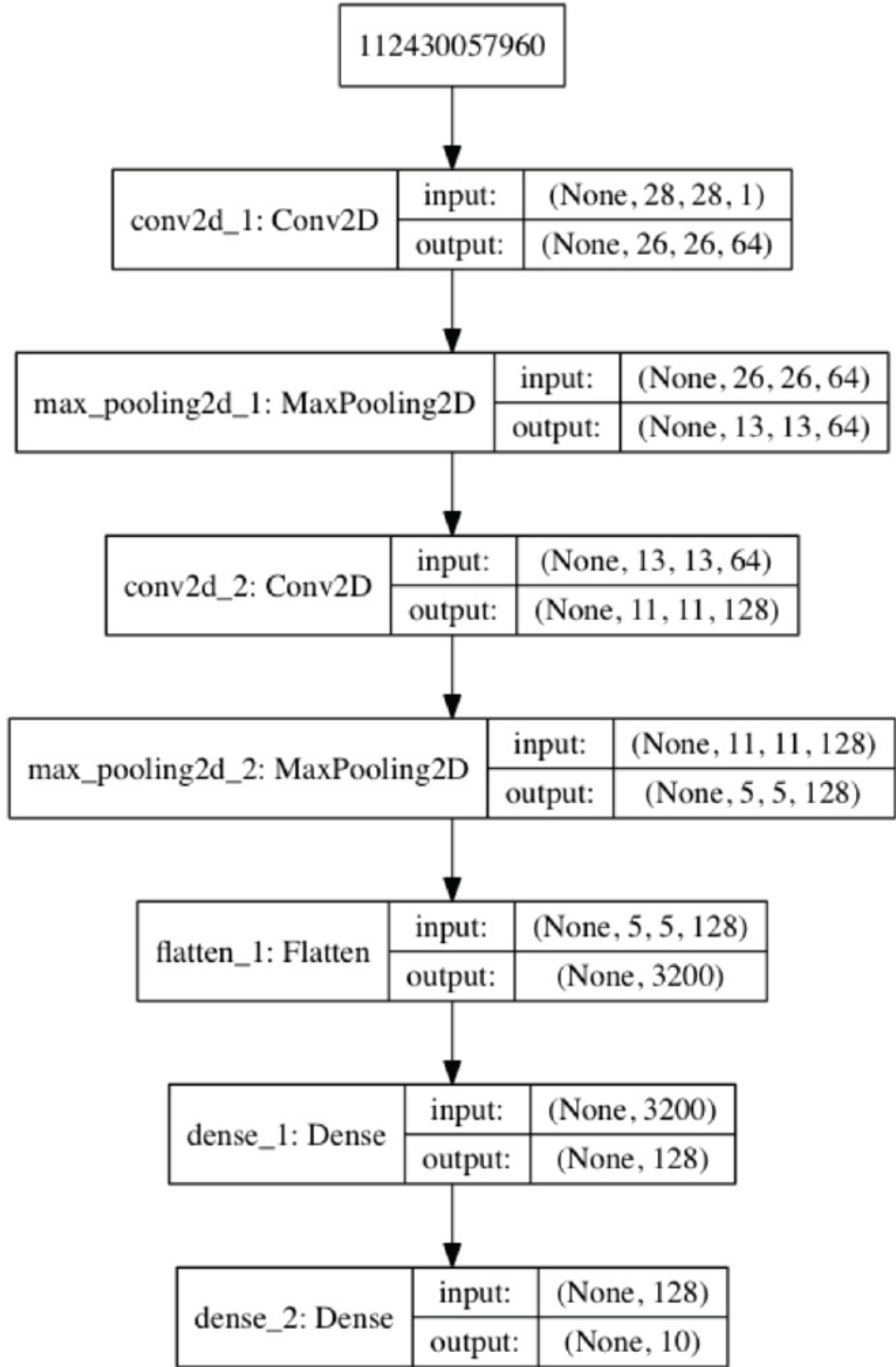


a bug in the current version of Keras. This node represents the input layer and should say InputLayer.



Compiling the Model

Once you've added all the layers you complete the model by calling its **compile method**:

[lick here to view code image](#)

```
[36]: cnn.compile(optimizer='adam',  
                 loss='categorical_crossentropy',  
                 metrics=['accuracy'])
```

The arguments are:

- `optimizer='adam'`—The *optimizer* this model will use to adjust the weights throughout the neural network as it learns. There are many optimizers⁰— 'adam' performs well across a wide variety of models.^{1, 2}

⁰For more Keras optimizers, see <https://keras.io/optimizers/>.

¹ <https://medium.com/octavian-ai/which-optimizer-and-learning-rate-should-i-use-for-deep-learning-5acb418f9b2>.

² <https://towardsdatascience.com/types-of-optimization-algorithms-used-in-neural-networks-and-ways-to-optimize-gradient-5ae5d39529f>.

- `loss='categorical_crossentropy'`—This is the *loss function* used by the optimizer in multi-classification networks like our convnet, which will predict 10 classes. As the neural network learns, the optimizer attempts to minimize the values returned by the loss function. The lower the loss, the better the neural network is at predicting what each image is. For binary classification (which we'll use later in this chapter), Keras provides 'binary_crossentropy', and for regression, 'mean_squared_error'. For other loss functions, see <https://keras.io/losses/>.
- `metrics=['accuracy']`—This is a list of the *metrics* that the network will produce to help you evaluate the model. Accuracy is a commonly used metric in classification models. In this example, we'll use the `accuracy` metric to check the percentage of correct predictions. For a list of other metrics, see <https://keras.io/metrics/>.

15.6.5 Training and Evaluating the Model

Similar to Scikit-learn's models, we train a Keras model by calling its **fit method**:

- As in Scikit-learn, the first two arguments are the training data and the categorical target labels.
- **epochs** specifies the number of times the model should process the entire set of training data. As we mentioned earlier, neural networks are trained iteratively.
- **batch_size** specifies the number of samples to process at a time during each epoch. Most models specify a power of 2 from 32 to 512. Larger batch sizes can decrease model accuracy.³ We chose 64. You can try different values to see how they affect the model's performance.

³Keskar, Nitish Shirish, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy and Ping Tak Peter Tang. On Large-Batch Training for Deep Learning: Generalization Gap

and Sharp Minima. CoRR abs/1609.04836 (2016).

<https://arxiv.org/abs/1609.04836>.

- In general, some samples should be used to *validate* the model. If you specify validation data, after each epoch, the model will use it to make predictions and display the *validation loss and accuracy*. You can study these values to tune your layers and the `fit` method's hyperparameters, or possibly change the layer composition of your model. Here, we used the **`validation_split`** argument to indicate that the model should reserve the *last* 10% (0.1) of the training samples for validation ⁴—in this case, 6000 samples will be used for validation. If you have separate validation data, you can use the `validation_data` argument (as you'll see in section 15.9) to specify a tuple containing arrays of samples and target labels. In general, it's better to get *randomly selected validation data*. You can use scikit-learn's `train_test_split` function for this purpose (as we'll do later in this chapter), then pass the randomly selected data with the `validation_data` argument.

⁴ <https://keras.io/getting-started/faq/#how-is-the-validation-split-computed>.

In the following output, we highlighted the training accuracy (`acc`) and validation accuracy (`val_acc`) in bold:

[lick here to view code image](#)

```
[37]: cnn.fit(X_train, y_train, epochs=5, batch_size=64,
            validation_split=0.1)
Train on 54000 samples, validate on 6000 samples
Epoch 1/5
54000/54000 [=====] - 68s 1ms/step - loss: 0.1407 -
poch 2/5
54000/54000 [=====] - 64s 1ms/step - loss: 0.0426 -
poch 3/5
54000/54000 [=====] - 69s 1ms/step - loss: 0.0299 -
poch 4/5
54000/54000 [=====] - 70s 1ms/step - loss: 0.0197 -
poch 5/5
54000/54000 [=====] - 63s 1ms/step - loss: 0.0155 -
[37]: <tensorflow.python.keras.callbacks.History at 0x7f105ba0ada0>
```

In section 15.7, we'll introduce *TensorBoard*—a TensorFlow tool for visualizing data from your deep-learning models. In particular, we'll view charts showing how the training and validation accuracy and loss values change through the epochs. In section 15.8, we'll demonstrate Andrej Karpathy's ConvnetJS tool, which trains convnets in your web browser and dynamically visualizes the layers' outputs, including what each convolutional layer "sees" as it learns. Also run his MNIST and CIFAR10 models. These will help you better understand neural networks' complex operations.

As the training proceeds, the `fit` method outputs information showing you the progress of each epoch, how long the epoch took to execute (in this case, each took 63–70 seconds), and the evaluation metrics for that pass. During the last epoch of this model, the accuracy reached

99.48% for the training samples (`acc`) and 99.27% for the validation samples (`val_acc`). Those are impressive numbers, given that we have not yet tried to tune the hyperparameters or tweak the number and types of the layers, which could lead to even better (or worse) results. Like machine learning, deep learning is an empirical science that benefits from lots of experimentation.

Evaluating the Model

Now we can check the accuracy of the model on data the model has not yet seen. To do so, we call the model's `evaluate` method, which displays as its output, how long it took to process the test samples (four seconds and 366 microseconds in this case):

[lick here to view code image](#)

```
[38]: loss, accuracy = cnn.evaluate(X_test, y_test)
10000/10000 [=====] - 4s   366us/step

[39]: loss
[39]: 0.026809450998473768

[40]: accuracy
[40]: 0.9917
```

According to the preceding output, our convnet model is 99.17% accurate when predicting the labels for unseen data—and, at this point, we have not tried to tune the model. With a little online research, you can find models that can predict MNIST with nearly 100% accuracy. Try experimenting with different numbers of layers, types of layers and layer parameters and observe how those changes affect your results.

Making Predictions

The model's `predict` method predicts the classes of the digit images in its argument array (`X_test`):

[lick here to view code image](#)

```
[41]: predictions = cnn.predict(X_test)
```

We can check what the first sample digit should be by looking at `y_test[0]`:

[lick here to view code image](#)

```
[42]: y_test[0]
[42]: array([0., 0., 0., 0., 0., 0., 0., 1., 0., 0.], dtype=float32)
```

According to this output, the first sample is the digit 7, because the categorical representation of the test sample's label specifies a 1.0 at index 7—recall that we created this representation via *one-hot encoding*.

Let's check the probabilities returned by the `predict` method for the first test sample:

[lick here to view code image](#)

```
[43]: for index, probability in enumerate(predictions[0]):
        print(f'{index}: {probability:.10%}')
0: 0.0000000201%
1: 0.0000001355%
2: 0.0000186951%
3: 0.0000015494%
4: 0.0000000003%
5: 0.0000000012%
6: 0.0000000000%
7: 99.9999761581%
8: 0.0000005577%
9: 0.0000011416%
```

According to the output, `predictions[0]` indicates that our model believes this digit is a 7 with *nearly* 100% certainty. Not all predictions have this level of certainty.

Locating the Incorrect Predictions

Next, we'd like to view some of the *incorrectly* predicted images to get a sense of the ones our model has trouble with. For example, if it's always mispredicting 8s, perhaps we need more 8s in our training data.

Before we can view incorrect predictions, we need to locate them. Consider `predictions[0]` above. To determine whether the prediction was correct, we must compare the index of the largest probability in `predictions[0]` to the index of the element containing 1.0 in `y_test[0]`. If these index values are the same, then the prediction was correct; otherwise, it was incorrect. NumPy's `argmax` function determines the index of the highest valued element in its array argument. Let's use that to locate the incorrect predictions. In the following snippet, `p` is the predicted value array, and `e` is the expected value array (the expected values are the labels for the dataset's test images):

[lick here to view code image](#)

```
[44]: images = X_test.reshape((10000, 28, 28))
        incorrect_predictions = []

        for i, (p, e) in enumerate(zip(predictions, y_test)):
            predicted, expected = np.argmax(p), np.argmax(e)

            if predicted != expected:
                incorrect_predictions.append(
                    (i, images[i], predicted, expected))
```

In this snippet, we first reshape the samples from the shape `(28, 28, 1)` that Keras required for learning back to `(28, 28)`, which Matplotlib requires to display the images. Next, we populate the list `incorrect_predictions` using the `for` statement. We `zip` the rows that represent each sample in the arrays `predictions` and `y_test`, then `enumerate` those so we can capture their indexes. If the `argmax` results for `p` and `e` are different, then the prediction was incorrect, and we append a tuple to `incorrect_predictions` containing that sample's index, image, the predicted value and the expected value. We

can confirm the total number of incorrect predictions (out of 10,000 images in the test set) with:

```
[45]: len(incorrect_predictions)
[45]: 83
```

Visualizing Incorrect Predictions

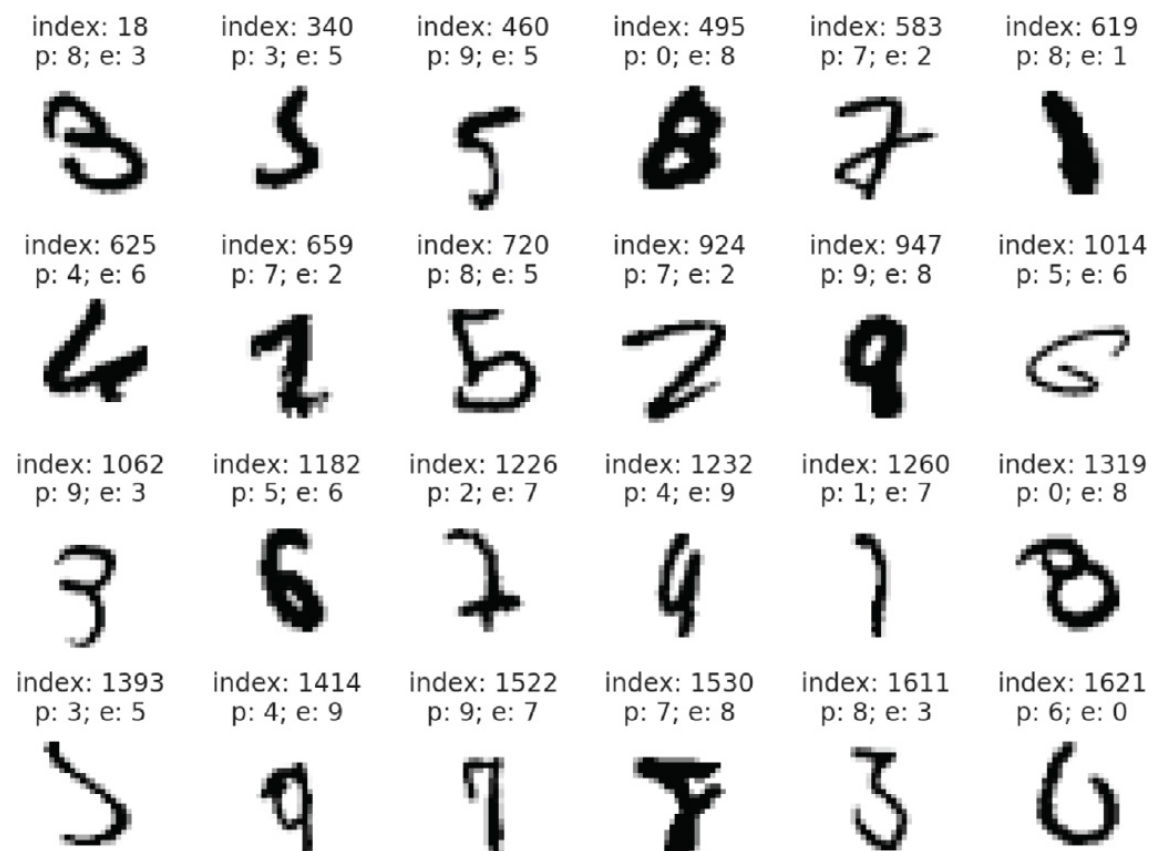
The following snippet displays 24 of the incorrect images labeled with each image's index, predicted value (p) and expected value (e):

[lick here to view code image](#)

```
[46]: figure, axes = plt.subplots(nrows=4, ncols=6, figsize=(16, 12))

for axes, item in zip(axes.ravel(), incorrect_predictions):
    index, image, predicted, expected = item
    axes.imshow(image, cmap=plt.cm.gray_r)
    axes.set_xticks([]) # remove x-axis tick marks
    axes.set_yticks([]) # remove y-axis tick marks
    axes.set_title(
        f'index: {index}\np: {predicted}; e: {expected}')
plt.tight_layout()
```

Before reading the expected values, look at each digit and write down what digit you think it is. This is an important part of getting to know your data:



Displaying the Probabilities for Several Incorrect Predictions

Let's look at the probabilities of some incorrect predictions. The following function displays the probabilities for the specified prediction array:

[lick here to view code image](#)

```
[47]: def display_probabilities(prediction):  
      for index, probability in enumerate(prediction):  
          print(f'{index}: {probability:.10%}')
```

Though the 8 (at index 495) in the first line of the image output looks like an 8, our model had trouble with it. As you can see in the following output, the model predicted this image as a 0, but also thought there was 16% chance it was a 6 and a 23% chance it was an 8:

[lick here to view code image](#)

```
[48]: display_probabilities(predictions[495])  
0: 59.7235262394%  
1: 0.0000015465%  
2: 0.8047289215%  
3: 0.0001740813%  
4: 0.0016636326%  
5: 0.0030567855%  
6: 16.1390662193%  
7: 0.0000001781%  
8: 23.3022540808%  
9: 0.0255270657%
```

The 2 (at index 583) in the first row was predicted to be a 7 with 62.7% certainty, but the model also thought there was a 36.4% chance it was a 2:

[lick here to view code image](#)

```
[49]: display_probabilities(predictions[583])  
0: 0.0000003016%  
1: 0.0000005715%  
2: 36.4056706429%  
3: 0.0176281916%  
4: 0.0000561930%  
5: 0.0000000003%  
6: 0.0000000019%  
7: 62.7455413342%  
8: 0.8310816251%  
9: 0.0000114385%
```

The 6 (at index 625) at the beginning of the second row was predicted to be a 4, though that was far from certain. In this case, the probability of a 4 (51.6%) was only slightly higher than the probability of a 6 (48.38%):

[lick here to view code image](#)

```
[50]: display_probabilities(predictions[625])  
0: 0.0008245181%  
1: 0.0000041209%  
2: 0.0012774357%  
3: 0.0000000009%  
4: 51.6223073006%  
5: 0.0000001779%  
6: 48.3754962683%
```

```
7: 0.0000000085%
8: 0.0000048182%
9: 0.0000785786%
```

15.6.6 Saving and Loading a Model

Neural network models can require significant training time. Once you've designed and tested a model that suits your needs, you can save its state. This allows you to load it later to make more predictions. Sometimes models are loaded and further trained for new problems. For example, layers in our model already know how to recognize features such as lines and curves, which could be useful in handwritten character recognition (as in the EMNIST dataset) as well. So you could potentially load the existing model and use it as the basis for a more robust model. This process is called **transfer learning**^{5, 6}—you transfer an existing model's knowledge into a new model. A Keras model's **save method** stores the model's architecture and state information in a format called **Hierarchical Data Format (HDF5)**. Such files use the `.h5` file extension by default:

⁵ <https://towardsdatascience.com/transfer-learning-from-pre-trained-models-f2393f124751>.

⁶ <https://medium.com/nanonets/nanonets-how-to-use-deep-learning-when-you-have-limited-data-f68c0b512cab>.

```
[51]: cnn.save('mnist_cnn.h5')
```

You can load a saved model with the **load_model function** from the `tensorflow.keras.models` module:

[lick here to view code image](#)

```
from tensorflow.keras.models import load_model
cnn = load_model('mnist_cnn.h5')
```

You can then invoke its methods. For example, if you've acquired more data, you could call `predict` to make additional predictions on new data, or you could call `fit` to start training with the additional data.

Keras provides several additional functions that enable you to save and load various aspects of your models. For more information, see

<https://keras.io/getting-started/faq/#how-can-i-save-a-keras-model>

15.7 VISUALIZING NEURAL NETWORK TRAINING WITH TENSORBOARD

With deep learning networks, there's so much complexity and so much going on internally that's hidden from you that it's difficult to know and fully understand all the details. This creates challenges in testing, debugging and updating models and algorithms. Deep learning

learns the features but there may be enormous numbers of them, and they may not be apparent to you.

Google provides the **TensorBoard**^{7, 8} tool for visualizing neural networks implemented in TensorFlow and Keras. Just as a car's dashboard visualizes data from your car's sensors, such as your speed, engine temperature and the amount of gas remaining, a **TensorBoard dashboard** visualizes data from a deep learning model that can give you insights into how well your model is learning and potentially help you tune its hyperparameters. Here, we'll introduce TensorBoard.

⁷ <https://github.com/tensorflow/tensorboard/blob/master/README.md>.

⁸ https://www.tensorflow.org/guide/summaries_and_tensorboard.

Executing TensorBoard

TensorBoard monitors a folder on your system looking for files containing the data it will visualize in a web browser. Here, you'll create that folder, execute the TensorBoard server, then access it via a web browser. Perform the following steps:

1. Change to the `ch15` folder in your Terminal, shell or Anaconda Command Prompt.
2. Ensure that your custom Anaconda environment `tf_env` is activated:

```
conda activate tf_env
```

3. Execute the following command to create a subfolder named `logs` in which your deep-learning models will write the information that TensorBoard will visualize:

```
mkdir logs
```

4. Execute TensorBoard

```
tensorboard --logdir=logs
```

5. You can now access TensorBoard in your web browser at

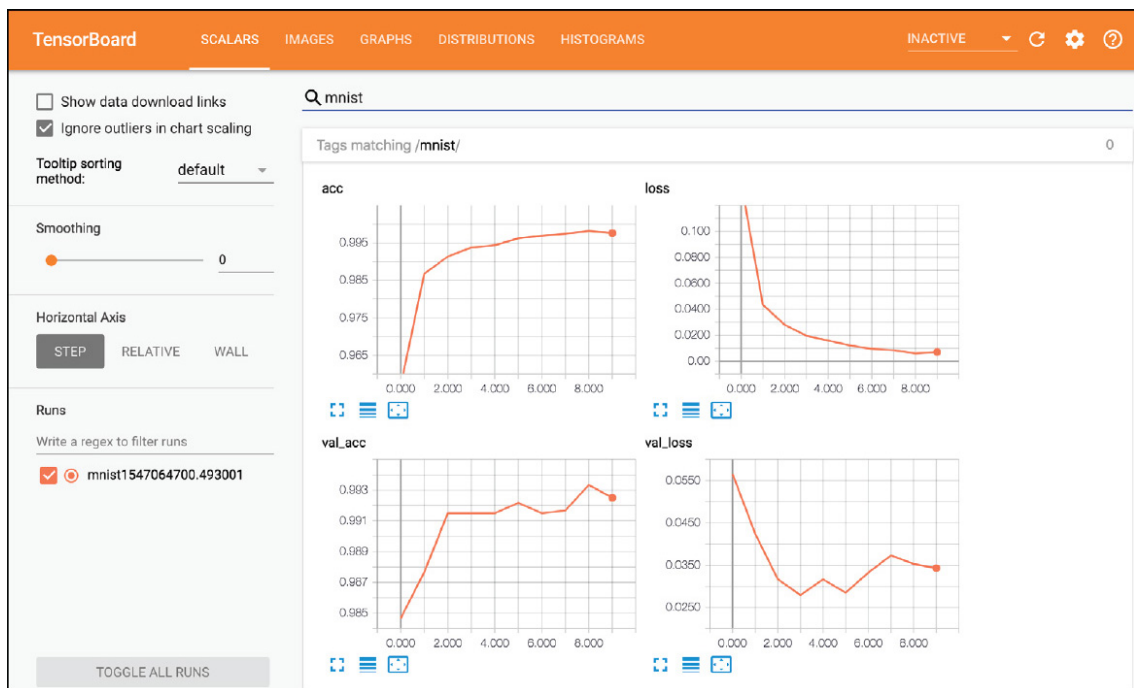
```
http://localhost:6006
```

If you connect to TensorBoard before executing any models, it will initially display a page indicating “No dashboards are active for the current data set.”⁹

⁹TensorBoard does not currently work with Microsofts Edge browser.

The TensorBoard Dashboard

TensorBoard monitors the folder you specified looking for files output by the model during training. When TensorBoard sees updates, it loads the data into the dashboard:



You can view the data as you train or after training completes. The dashboard above shows the TensorBoard **SCALARS** tab, which displays charts for individual values that change over time, such as the training accuracy (`acc`) and training loss (`loss`) shown in the first row, and the validation accuracy (`val_acc`) and validation loss (`val_loss`) shown in the second row. The diagrams visualize a 10-epoch run of our MNIST convnet, which we provided in the notebook `MNIST_CNN_TensorBoard.ipynb`. The epochs are displayed along the *x*-axes starting from 0 for the first epoch. The accuracy and loss values are displayed on the *y*-axes. Looking at the training and validation accuracies, you can see in the first 5 epochs similar results to the five-epoch run in the previous section.

For the 10-epoch run, the training accuracy continued to improve through the 9th epoch, then decreased slightly. This might be the point at which we're starting to overfit, but we might need to train longer to find out. For the validation accuracy, you can see that it jumped up quickly, then was relatively flat for five epochs before jumping up then decreasing. For the training loss, you can see that it drops quickly, then continuously declines through the ninth epoch, before a slight increase. The validation loss dropped quickly then bounced around. We could run this model for more epochs to see whether results improve, but based on these diagrams, it appears that around the sixth epoch we get a nice combination of training and validation accuracy with minimal validation loss.

Normally these diagrams are stacked vertically in the dashboard. We used the search field (above the diagrams) to show any that had the name "mnist" in their folder name—we'll configure that in a moment. TensorBoard can load data from multiple models at once and you can choose which to visualize. This makes it easy to compare several different models or multiple runs of the same model.

Copy the MNIST Convnet's Notebook

To create the new notebook for this example:

1. Right-click the `MNIST_CNN.ipynb` notebook in JupyterLab's **File Browser** tab and select **Duplicate** to make a copy of the notebook.

2. Right-click the new notebook named `MNIST_CNN-Copy1.ipynb`, then select **Rename**, enter the name `MNIST_CNN_TensorBoard.ipynb` and press *Enter*.

Open the notebook by double-clicking its name.

Configuring Keras to Write the TensorBoard Log Files

To use TensorBoard, before you `fit` the model, you need to configure a **TensorBoard** object (module `tensorflow.keras.callbacks`), which the model will use to write data into a specified folder that TensorBoard monitors. This object is known as a **callback** in Keras. In the notebook, click to the left of snippet that calls the model's `fit` method, then type `a`, which is the shortcut for adding a new code cell *above* the current cell (use `b` for *below*). In the new cell, enter the following code to create the `TensorBoard` object:

[lick here to view code image](#)

```
from tensorflow.keras.callbacks import TensorBoard
import time

tensorboard_callback = TensorBoard(log_dir=f'./logs/mnist{time.time()}',
                                   histogram_freq=1, write_graph=True)
```

The arguments are:

- `log_dir`—The name of the folder in which this model's log files will be written. The notation `'./logs/'` indicates that we're creating a new folder within the logs folder you created previously, and we follow that with `'mnist'` and the current time. This ensures that each new execution of the notebook will have its own log folder. That will enable you to compare multiple executions in TensorBoard.
- `histogram_freq`—The frequency in *epochs* that Keras will output to the model's log files. In this case, we'll write data to the logs for every epoch.
- `write_graph`—When this is true, a graph of the model will be output. You can view the graph in the **GRAPHS** tab in TensorBoard.

Updating Our Call to `fit`

Finally, we need to modify the original `fit` method call in snippet 37. For this example, we set the number of epochs to 10, and we added the `callbacks` argument, which is a list of callback objects ^o:

^oYou can view Keras other callbacks at <https://keras.io/callbacks/>.

[lick here to view code image](#)

```
cnn.fit(X_train, y_train, epochs=10, batch_size=64,
        validation_split=0.1, callbacks=[tensorboard_callback])
```

You can now re-execute the notebook by selecting **Kernel > Restart Kernel and Run All**

Cells in JupyterLab. After the first epoch completes, you'll start to see data in TensorBoard.

15.8 CONVNETJS: BROWSER-BASED DEEP-LEARNING TRAINING AND VISUALIZATION

In this section, we'll overview Andrej Karpathy's JavaScript-based ConvnetJS tool for training and visualizing convolutional neural networks in your web browser: ¹

¹You also can download ConvnetJS from GitHub at
`https://github.com/karpathy/convnetjs`.

[lick here to view code image](#)

```
https://cs.stanford.edu/people/karpathy/convnetjs/
```

You can run the ConvnetJS sample convolutional neural networks or create your own. We've used the tool on several desktop, tablet and phone browsers.

The ConvnetJS MNIST demo trains a convolutional neural network using the MNIST dataset we presented in [ection 15.6](#). The demo presents a scrollable dashboard that updates dynamically as the model trains and contains several sections.

Training Stats

This section contains a **Pause** button that enables you to stop the learning and “freeze” the current dashboard visualizations. Once you pause the demo, the button text changes to **resume**. Clicking the button again continues training. This section also presents training statistics, including the training and validation accuracy and a graph of the training loss.

Instantiate a Network and Trainer

In this section, you'll find the JavaScript code that creates the convolutional neural network. The default network has similar layers to the convnet in [ection 15.6](#). The Conv-netJS documentation ² shows the supported layer types and how to configure them. You can experiment with different layer configurations in the provided textbox and begin training an updated network by clicking the **change network** button.

² `https://cs.stanford.edu/people/karpathy/convnetjs/docs.html`.

Network Visualization

This key section shows one training image at a time and how the network processes that image through each layer. Click the **Pause** button to inspect all the layers' outputs for a given digit to get a sense of what the network “sees” as it learns. The network's last layer produces the probabilistic classifications. It shows 10 squares—9 black and 1 white, indicating the predicted class of the current digit image.

Example Predictions on Test Set

The final section shows a random selection of the test set images and the top three possible classes for each digit. The one with the highest probability is shown on a green bar and the

other two are displayed on red bars. The length of each bar is a visual indication of that class's probability.

15.9 RECURRENT NEURAL NETWORKS FOR SEQUENCES; SENTIMENT ANALYSIS WITH THE IMDB DATASET

In the MNIST CNN network, we focused on stacked layers that were applied *sequentially*. Non-sequential models are possible, as you'll see here with *recurrent neural networks*. In this section, we use Keras's bundled IMDB (the Internet Movie Database) movie reviews dataset ³ to perform **binary classification**, predicting whether a given review's sentiment is positive or negative.

³Maas, Andrew L. and Daly, Raymond E. and Pham, Peter T. and Huang, Dan and Ng, Andrew Y. and Potts, Christopher, "Learning Word Vectors for Sentiment Analysis," *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, June 2011. Portland, Oregon, USA. Association for Computational Linguistics, pp. 142150. <http://www.aclweb.org/anthology/P11-015>.

We'll use a **recurrent neural network (RNN)**, which processes sequences of data, such as time series or text in sentences. The term "recurrent" comes from the fact that the neural network contains *loops* in which the output of a given layer becomes the input to that same layer in the next **time step**. In a time series, a time step is the next point in time. In a text sequence, a "time step" would be the next word in a sequence of words.

The looping in RNNs enables them to learn and remember relationships among the data in the sequence. For example, consider the following sentences we used in the "Natural Language Processing" chapter. The sentence

```
The food is not good.
```

clearly has negative sentiment. Similarly, the sentence

```
The movie was good.
```

has positive sentiment, though not as positive as

```
The movie was excellent!
```

In the first sentence, the word "good" on its own has positive sentiment. However, when *preceded by* "not," which appears earlier in the sequence, the sentiment becomes negative. RNNs take into account the relationships among the earlier and later parts of a sequence.

In the preceding example, the words that determined sentiment were adjacent. However, when determining the meaning of text there can be many words to consider and an arbitrary number of words in between them. In this section, we'll use a **Long Short-Term Memory (LSTM)** layer, which makes the neural network *recurrent* and is optimized to handle

learning from sequences like the ones we described above.

RNNs have been used for many tasks including: ^{4, 5, 6}

⁴ <https://www.analyticsindiamag.com/overview-of-recurrent-neural-networks-and-their-applications/>.

⁵ https://en.wikipedia.org/wiki/Recurrent_neural_network#Applications.

⁶ <http://karpathy.github.io/2015/05/21/rnn-effectiveness/>.

- predictive text input—displaying possible next words as you type,
- sentiment analysis,
- responding to questions with the predicted best answers from a corpus,
- inter-language translation, and
- automated closed captioning in video.

15.9.1 Loading the IMDb Movie Reviews Dataset

The IMDb movie reviews dataset included with Keras contains 25,000 training samples and 25,000 testing samples, each labeled with its positive (1) or negative (0) sentiment. Let's import the `tensorflow.keras.datasets.imdb` module so we can load the dataset:

[lick here to view code image](#)

```
[1]: from tensorflow.keras.datasets import imdb
```

The `imdb` module's `load_data` function returns the IMDb training and testing sets. There are over 88,000 unique words in the dataset. The `load_data` function enables you to specify the number of unique words to import as part of the training and testing data. In this case, we loaded only the top 10,000 most frequently occurring words due to the memory limitations of our system and the fact that we're (intentionally) training on a CPU rather than a GPU (because most of our readers will not have access to systems with GPUs and TPUs). The more data you load, the longer training will take, but more data may help produce better models:

[lick here to view code image](#)

```
[2]: number_of_words = 10000

[3]: (X_train, y_train), (X_test, y_test) = imdb.load_data(
    num_words=number_of_words)
```

The `load_data` function returns a tuple of two elements containing the training and testing sets. Each element is itself a tuple containing the samples and labels, respectively. In a given review, `load_data` replaces any words outside the top 10,000 with a placeholder value, which we'll discuss shortly.

15.9.2 Data Exploration

Let's check the dimensions of the training set samples (`X_train`), training set labels (`y_train`), testing set samples (`X_test`) and testing set labels (`y_test`):

[click here to view code image](#)

```
[4]: X_train.shape
[4]: (25000,)

[5]: y_train.shape
[5]: (25000,)

[6]: X_test.shape
[6]: (25000,)

[7]: y_test.shape
[7]: (25000,)
```

The arrays `y_train` and `y_test` are one-dimensional arrays containing 1s and 0s, indicating whether each review is positive or negative. Based on the preceding outputs, `X_train` and `X_test` also appear to be one-dimensional. However, their elements actually are *lists* of integers, each representing one review's contents, as shown in snippet [9]:⁷

⁷Here we used the `%pprint` magic to turn off pretty printing so the following snippets output could be displayed horizontally rather than vertically to save space. You can turn pretty printing back on by re-executing the `%pprint` magic.

[click here to view code image](#)

```
[8]: %pprint
[8]: Pretty printing has been turned OFF

[9]: X_train[123]
[9]: [1, 307, 5, 1301, 20, 1026, 2511, 87, 2775, 52, 116, 5, 31, 7, 4, 91, 12
```

Keras deep learning models require *numeric data*, so the Keras team preprocessed the IMDb dataset for you.

Movie Review Encodings

Because the movie reviews are numerically encoded, to view their original text, you need to know the word to which each number corresponds. Keras's IMDb dataset provides a dictionary that maps the words to their indexes. Each word's corresponding value is its frequency ranking among all the words in the entire set of reviews. So the word with the ranking 1 is the most frequently occurring word (calculated by the Keras team from the dataset), the word with ranking 2 is the second most frequently occurring word, and so on.

Though the dictionary values begin with 1 as the most frequently occurring word, in each encoded review (like `X_train[123]` shown previously), the ranking values are *offset by 3*. So any review containing the most frequently occurring word will have the value 4 wherever that word appears in the review. Keras reserves the values 0, 1 and 2 in each encoded review

for the following purposes:

- The value 0 in a review represents *padding*. Keras deep learning algorithms expect all the training samples to have the same dimensions, so some reviews may need to be expanded to a given length and some shortened to that length. Reviews that need to be expanded are padded with 0s.
- The value 1 represents a token that Keras uses internally to indicate the start of a text sequence for learning purposes.
- The value 2 in a review represents an unknown word—typically a word that was not loaded because you called `load_data` with the `num_words` argument. In this case, any review that contained words with frequency rankings greater than `num_words` would have those words' numeric values replaced with 2. This is all handled by Keras when you load the data.

Because each review's numeric values are offset by 3, we'll have to account for this when we decode the review.

Decoding a Movie Review

Let's decode a review. First, get the word-to-index dictionary by calling the function `get_word_index` from the `tensorflow.keras.datasets.imdb` module:

[lick here to view code image](#)

```
[10]: word_to_index = imdb.get_word_index()
```

The word 'great' might appear in a positive movie review, so let's see whether it's in the dictionary:

[lick here to view code image](#)

```
[11]: word_to_index['great']  
[11]: 84
```

According to the output, 'great' is the dataset's 84th most frequent word. If you look up a word that's not in the dictionary, you'll get an exception.

To transform the frequency ratings into words, let's first reverse the `word_to_index` dictionary's mapping, so we can look up every word by its frequency rating. The following dictionary comprehension reverses the mapping:

[lick here to view code image](#)

```
[12]: index_to_word = \n      {index: word for (word, index) in word_to_index.items() }
```

Recall that a dictionary's `items` method enables us to iterate through tuples of key-value

pairs. We unpack each tuple into the variables `word` and `index`, then create an entry in the new dictionary with the expression `index: word`.

The following list comprehension gets the top 50 words from the new dictionary—recall that the most frequent word has the value 1:

[lick here to view code image](#)

```
[13]: [index_to_word[i] for i in range(1, 51)]
[13]: ['the', 'and', 'a', 'of', 'to', 'is', 'br', 'in', 'it', 'i', 'this', 't
```

ote that most of these are *stop words*. Depending on the application, you might want to remove or keep the stop words. For example, if you were creating a predictive-text application that suggests the next word in a sentence the user is typing, you'd want to keep the stop words so they can be displayed as predictions.

Now, we can decode a review. We use the `index_to_word` dictionary's two-argument method `get` rather than the `[]` operator to get value for each key. If a value is not in the dictionary, the `get` method returns its second argument, rather than raising an exception. The argument `i - 3` accounts for the offset in the encoded reviews of each review's frequency ratings. When the Keras reserved values 0–2 appear in a review, `get` returns '?'; otherwise, `get` returns the word with the key `i - 3` in the `index_to_word` dictionary:

[lick here to view code image](#)

```
[14]: ' '.join([index_to_word.get(i - 3, '?') for i in X_train[123]])
[14]: '? beautiful and touching movie rich colors great settings good
      acting and one of the most charming movies i have seen in a while i
      never saw such an interesting setting when i was in china my wife
      liked it so much she asked me to ? on and rate it so other would
      enjoy too'
```

We can see from the `y_train` array that this review is classified as positive:

```
[15]: y_train[123]
[15]: 1
```

15.9.3 Data Preparation

The number of words per review varies, but the Keras requires all samples to have the same dimensions. So, we need to perform some data preparation. In this case, we need to restrict every review to the *same* number of words. Some reviews will need to be *padded* with additional data and others will need to be *truncated*. The `pad_sequences` utility function (module `tensorflow.keras.preprocessing.sequence`) reshapes `X_train`'s samples (that is, its rows) to the number of features specified by the `maxlen` argument (200) and returns a two-dimensional array:

[lick here to view code image](#)

```
[16]: words_per_review = 200

[17]: from tensorflow.keras.preprocessing.sequence import pad_sequences

[18]: X_train = pad_sequences(X_train, maxlen=words_per_review)
```

If a sample has more features, `pad_sequences` truncates it to the specified length. If a sample has fewer features, `pad_sequences` adds 0s to the beginning of the sequence to pad it to the specified length. Let's confirm `X_train`'s new shape:

```
[19]: X_train.shape
[19]: (25000, 200)
```

We also must reshape `X_test` for later in this example when we evaluate the model:

[lick here to view code image](#)

```
[20]: X_test = pad_sequences(X_test, maxlen=words_per_review)

[21]: X_test.shape
[21]: (25000, 200)
```

Splitting the Test Data into Validation and Test Data

In our convnet, we used the `fit` method's `validation_split` argument to indicate that 10% of our training data should be set aside to validate the model as it trains. For this example, we'll manually split the 25,000 test samples into 20,000 test samples and 5,000 validation samples. We'll then pass the 5,000 validation samples to the model's `fit` method via the argument `validation_data`. Let's use Scikit-learn's `train_test_split` function from the previous chapter to split the test set:

[lick here to view code image](#)

```
[22]: from sklearn.model_selection import train_test_split
      X_test, X_val, y_test, y_val = train_test_split(
          X_test, y_test, random_state=11, test_size=0.20)
```

Let's also confirm the split by checking `X_test`'s and `X_val`'s shapes:

```
[23]: X_test.shape
[23]: (20000, 200)

[24]: X_val.shape
[24]: (5000, 200)
```

15.9.4 Creating the Neural Network

Next, we'll configure the RNN. Once again, we begin with a `Sequential` model to which we'll add the layers that compose our network:

[lick here to view code image](#)

```
[25]: from tensorflow.keras.models import Sequential

[26]: rnn = Sequential()
```

Next, let's import the layers we'll use in this model:

[lick here to view code image](#)

```
[27]: from tensorflow.keras.layers import Dense, LSTM

[28]: from tensorflow.keras.layers.embeddings import Embedding
```

Adding an Embedding Layer

Previously, we used *one-hot encoding* to convert the MNIST dataset's integer labels into *categorical* data. The result for each label was a vector in which all but one element was 0. We could do that for the index values that represent our words. However, this example processes 10,000 unique words. That means we'd need a 10,000-by-10,000 array to represent all the words. That's 100,000,000 elements, and almost all the array elements would be 0. This is not an efficient way to encode the data. If we were to process all 88,000+ unique words in the dataset, we'd need an array of nearly *eight billion* elements!

To *reduce dimensionality*, RNNs that process text sequences typically begin with an **embedding layer** that encodes each word in a more compact *dense-vector representation*. The vectors produced by the embedding layer also capture the word's context—that is, how a given word relates to the words around it. So the embedding layer enables the RNN to learn word relationships among the training data.

There are also **predefined word embeddings**, such as **Word2Vec** and **GloVe**. You can load these into neural networks to save training time. They're also sometimes used to add basic word relationships to a model when smaller amounts of training data are available. This can improve the model's accuracy by allowing it to build upon previously learned word relationships, rather than trying to learn those relationships with insufficient amounts of data.

Let's create an **Embedding** layer (module `tensorflow.keras.layers`):

[lick here to view code image](#)

```
[29]: rnn.add(Embedding(input_dim=number_of_words, output_dim=128,
                        input_length=words_per_review))
```

The arguments are:

- `input_dim`—The number of unique words.
- `output_dim`—The size of each word embedding. If you load pre-existing embeddings ⁸

like *Word2Vec* and *GloVe*, you must set this to match the size of the word embeddings you load.

⁸ <https://blog.keras.io/using-pre-trained-word-embeddings-in-a-keras-model.html>.

- `input_length=words_per_review`—The number of words in each input sample.

Adding an LSTM Layer

Next, we'll add an LSTM layer:

[lick here to view code image](#)

```
[30]: rnn.add(LSTM(units=128, dropout=0.2, recurrent_dropout=0.2))
```

The arguments are:

- `units`—The number of neurons in the layer. The more neurons the more the network can remember. As a guideline, you can start with a value between the length of the sequences you're processing (200 in this example) and the number of classes you're trying to predict (2 in this example).⁹

⁹ <https://towardsdatascience.com/choosing-the-right-hyperparameters-for-a-simple-lstm-using-keras-f8e9ed76f046>.

- `dropout`—The percentage of neurons to randomly disable when processing the layer's input and output. Like the pooling layers in our convnet, **dropout** is a proven technique^{0, 1} that reduces overfitting. Keras provides a **Dropout** layer that you can add to your models.

⁰Yarin, Ghahramani, and Zoubin. A Theoretically Grounded Application of Dropout in Recurrent Neural Networks. October 05, 2016.

<https://arxiv.org/abs/1512.05287>.

¹Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research* 15 (June 14, 2014): 1929-1958.

<http://jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf>.

- `recurrent_dropout`—The percentage of neurons to randomly disable when the layer's output is fed back into the layer again to allow the network to learn from what it has seen previously.

The mechanics of how the LSTM layer performs its task are beyond the scope of this book. Chollet says: “you don't need to understand anything about the specific architecture of an LSTM cell; as a human, it shouldn't be your job to understand it. Just keep in mind what the LSTM cell is meant to do: allow past information to be reinjected at a later time.”²

Adding a Dense Output Layer

Finally, we need to take the LSTM layer's output and reduce it to one result indicating whether a review is positive or negative, thus the value 1 for the `units` argument. Here we use the '**sigmoid**' **activation function**, which is preferred for binary classification.³ It reduces arbitrary values into the range 0.0–1.0, producing a probability:

³Chollet, François. *Deep Learning with Python*. p.114. Shelter Island, NY: Manning Publications, 2018.

[lick here to view code image](#)

```
[31]: rnn.add(Dense(units=1, activation='sigmoid'))
```

Compiling the Model and Displaying the Summary

Next, we compile the model. In this case, there are only two possible outputs, so we use the **binary_crossentropy loss function**:

[lick here to view code image](#)

```
[32]: rnn.compile(optimizer='adam',
                  loss='binary_crossentropy',
                  metrics=['accuracy'])
```

The following is the summary of our model. Notice that even though we have fewer layers than our convnet, the RNN has nearly three times as many trainable parameters (the network's weights) as the convnet and more parameters means more training time. The large number of parameters primarily comes from the number of words in the vocabulary (we loaded 10,000) times the number of neurons in the `Embedding` layer's output (128):

[lick here to view code image](#)

```
[33]: rnn.summary()
```

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 200, 128)	1280000
lstm_1 (LSTM)	(None, 128)	131584
dense_1 (Dense)	(None, 1)	129

=====
Total params: 1,411,713
Trainable params: 1,411,713
Non-trainable params: 0

15.9.5 Training and Evaluating the Model

Let's train our model. ⁴ Notice for each epoch that the model takes significantly longer to train than our convnet did. This is due to the larger numbers of parameters (weights) our RNN model needs to learn. We bolded the accuracy (`acc`) and validation accuracy (`val_acc`) values for readability—these represent the percentage of training samples and the percentage of `validation_data` samples that the model predicts correctly.

⁴At the time of this writing, TensorFlow displayed a warning when we executed this statement. This is a known TensorFlow issue and, according to the forums, you can safely ignore the warning.

[lick here to view code image](#)

```
[34]: rnn.fit(X_train, y_train, epochs=10, batch_size=32,
            validation_data=(X_test, y_test))
Train on 25000 samples, validate on 5000 samples
Epoch 1/5
25000/25000 [=====] - 299s   12ms/step - loss: 0.6574
poch 2/5
25000/25000 [=====] - 298s   12ms/step - loss: 0.4577
poch 3/5
25000/25000 [=====] - 296s   12ms/step - loss: 0.3277
poch 4/5
25000/25000 [=====] - 307s   12ms/step - loss: 0.2675
poch 5/5
25000/25000 [=====] - 310s   12ms/step - loss: 0.2217
[34]: <tensorflow.python.keras.callbacks.History object at 0xb3ba882e8>
```

Finally, we can evaluate the results using the test data. Function `evaluate` returns the loss and accuracy values. In this case, the model was 85.99% accurate:

[lick here to view code image](#)

```
[35]: results = rnn.evaluate(X_test, y_test)
20000/20000 [=====] - 42s 2ms/step

[36]: results
[36]: [0.3415240607559681, 0.8599]
```

Note that the accuracy of this model seems low compared to our MNIST convnet's results, but this is a much more difficult problem. If you search online for other IMDB sentiment-analysis binary-classification studies, you'll find lots of results in the high 80s. So we did reasonably well with our small recurrent neural network of only three layers. You might want to study some online models and try to produce a better model.

15.10 TUNING DEEP LEARNING MODELS

In section 15.9.5, notice in the `fit` method's output that both the testing accuracy (85.99%) and validation accuracy (87.04%) were significantly less than the 90.83% training accuracy. Such disparities are usually the result of overfitting, so there is plenty of room for improvement in our model. ⁵ ⁶ If you look at the output of each epoch, you'll notice both the training and validation accuracy continue to increase. Recall that training for too many

pochs can lead to overfitting, but it's possible we have not yet trained enough. Perhaps one hyperparameter tuning option for this model would be to increase the number of epochs.

⁵ <https://towardsdatascience.com/deep-learning-overfitting-46bf5b35e24>.

⁶ <https://hackernoon.com/memorizing-is-not-learning-6-tricks-to-revent-overfitting-in-machine-learning-820b091dc42>.

Some variables that affect your models' performance include:

- having more or less data to train with
- having more or less to test with
- having more or less to validate with
- having more or fewer layers
- the types of layers you use
- the order of the layers

In our IMDB RNN example, some things we could tune include:

- trying different amounts of the training data—we used only the top 10,000 words
- different numbers of words per review—we used only 200,
- different numbers of neurons in our layers,
- more layers or
- possibly loading pre-trained word vectors rather than having our Embedding layer learn them from scratch.

The compute time required to train models multiple times is significant so, in deep learning, you generally do not tune hyperparameters with techniques like k-fold cross-validation or grid search. ⁷ There are various tuning techniques, ⁸, ⁹, ⁰, ¹ but one particularly promising area is automated machine learning (AutoML). For example, the Auto-Keras ² library is specifically geared to automatically choosing the best configurations for your Keras models. Google's Cloud AutoML and Baidu's EZDL are among various other automated machine learning efforts.

⁷ <https://www.quora.com/Is-cross-validation-heavily-used-in-deep-learning-or-is-it-too-expensive-to-be-used>.

⁸ <https://towardsdatascience.com/what-are-hyperparameters-and-how-to-une-the-hyperparameters-in-a-deep-neural-network-d0604917584a>.

⁹ <https://medium.com/machine-learning-bites/deeplearning-series-deep-neural-networks-tuning-and-optimization-39250ff7786d>.

⁰ <https://flyyufelix.github.io/2016/10/03/fine-tuning-in-keras-art1.html> and <https://flyyufelix.github.io/2016/10/08/fine-tuning-in-keras-part2.html>.

¹ <https://towardsdatascience.com/a-comprehensive-guide-on-how-to-fine-tune-deep-neural-networks-using-keras-on-google-colab-free-aaaa0aced8f>.

² <https://autokeras.com/>.

5.11 CONVNET MODELS PRETRAINED ON IMAGENET

With deep learning, rather than starting fresh on every project with costly training, validating and testing, you can use **pretrained deep neural network models** to:

- make new predictions,
- continue training them further with new data or
- transfer the weights learned by a model for a similar problem into a new model—this is called *transfer learning*.

Keras Pretrained Convnet Models

Keras comes bundled with the following pretrained convnet models, ³ each pretrained on Image-Net ⁴—a growing dataset of 14+ million images:

³ <https://keras.io/applications/>.

⁴ <http://www.image-net.org>.

- Xception
- VGG16
- VGG19
- ResNet50
- Inception v3
- Inception-ResNet v2
- MobileNet v1
- DenseNet
- NASNet

- MobileNet v2

Reusing Pretrained Models

ImageNet is too big for efficient training on most computers, so most people interested in using it start with one of the smaller pretrained models.

You can reuse just the architecture of each model and train it with new data, or you can reuse the pretrained weights. For a few simple examples, see:

```
https://keras.io/applications/
```

ImageNet Challenge

In the end-of-chapter projects, you'll research and use some of these bundled models. You'll also investigate the *ImageNet Large Scale Visual Recognition Challenge* for evaluating object-detection and image-recognition models.⁵ This competition ran from 2010 through 2017. ImageNet now has a continuously running challenge on the Kaggle competition site called the *ImageNet Object Localization Challenge*.⁶ The goal is to identify "all objects within an image, so those images can then be classified and annotated." ImageNet releases the current participants leaderboard once per quarter.

⁵ <http://www.image-net.org/challenges/LSVRC/>.

⁶ <https://www.kaggle.com/c/imagenet-object-localization-challenge>.

A lot of what you've seen in the machine learning and deep learning chapters is what the Kaggle competition website is all about. There's no obvious optimal solution for many machine learning and deep learning tasks. People's creativity is really the only limit. On Kaggle, companies and organizations fund competitions where they encourage people worldwide to develop better-performing solutions than they've been able to do for something that's important to their business or organization. Sometimes companies offer prize money, which has been as high as \$1,000,000 on the famous Netflix competition. Netflix wanted to get a 10% or better improvement in their model for determining whether people will like a movie, based on how they rated previous ones.⁷ They used the results to help make better recommendations to members. Even if you do not win a Kaggle competition, it's a great way to get experience working on problems of current interest.

⁷ <https://netflixprize.com/rules.html>.

15.12 WRAP-UP

In chapter 16, you peered into the future of AI. Deep Learning has captured the imagination of the computer-science and data science-communities. This may be the most important AI chapter in the book.

We mentioned the key deep-learning platforms, indicating that Google's TensorFlow is the most widely used. We discussed why Keras, which presents a friendly interface to TensorFlow, has become so popular.

e set up a custom Anaconda environment for TensorFlow, Keras and JupyterLab, then used the environment to implement the Keras examples.

We explained what tensors are and why they're crucial to deep learning. We discussed the basics of neurons and multi-layered neural networks for building Keras deep-learning models. We considered some popular types of layers and how to order them.

We introduced convolutional neural networks (convnets) and indicated that they're especially appropriate for computer-vision applications. We then built, trained, validated and tested a convnet using the MNIST database of handwritten digits for which we achieved 99.17% prediction accuracy. This is remarkable, given that we achieved it by working with a only a basic model and without doing any hyperparameter tuning. You can try more sophisticated models and tune the hyperparameters to try to achieve better performance. We listed a variety of intriguing computer vision tasks.

We introduced TensorBoard for visualizing TensorFlow and Keras neural network training and validation. We also discussed ConvnetJS, a browser-based convnet training and visualization tool, which enables you to peek inside the training process.

Next, we presented recurrent neural networks (RNNs) for processing sequences of data, such as time series or text in sentences. We used an RNN with the IMDB movie reviews dataset to perform binary classification, predicting whether each review's sentiment was positive or negative. We also discussed tuning deep learning models and how high-performance hardware, like NVIDIA's GPUs and Google's TPUs, is making it possible for more people to tackle more substantial deep-learning studies.

Given how costly and time-consuming it is to train deep-learning models, we explained the strategy of using pretrained models. We listed various Keras convnet image-processing models that were trained on the massive ImageNet dataset, and discussed how transfer learning enables you to use these models to create new ones quickly and effectively. Deep learning is a large, complex topic. We focused on the basics in the chapter.

In the next chapter, we present the big data infrastructure that supports the kinds of AI technologies we've discussed in chapters 12 through 15. We'll consider the Hadoop and Spark platforms for big data batch processing and real-time streaming applications. We'll look at relational databases and the SQL language for querying them—these have dominated the database field for many decades. We'll discuss how big data presents challenges that relational databases don't handle well, and consider how NoSQL databases are designed to handle those challenges. We'll conclude the book with a discussion of the Internet of Things (IoT), which will surely be the world's largest big-data source and will present many opportunities for entrepreneurs to develop leading-edge businesses that will truly make a difference in people's lives.



16. Big Data: Hadoop, Spark, NoSQL and IoT

Objectives

In this chapter you'll:

- Understand what big data is and how quickly it's getting bigger.
- Manipulate a SQLite relational database using Structured Query Language (SQL).
- Understand the four major types of NoSQL databases.
- Store tweets in a MongoDB NoSQL JSON document database and visualize them on a Folium map.
- Understand Apache Hadoop and how it's used in big-data batch-processing applications.
- Build a Hadoop MapReduce application on Microsoft's Azure HDInsight cloud service.
- Understand Apache Spark and how it's used in high-performance, real-time big-data applications.
- Use Spark streaming to process data in mini-batches.
- Understand the Internet of Things (IoT) and the publish/subscribe model.
- Publish messages from a simulated Internet-connected device and visualize its messages in a dashboard.
- Subscribe to PubNub's live Twitter and IoT streams and visualize the data.

outline

6.1 Introduction

6.2 Relational Databases and Structured Query Language (SQL)

6.3.1 A books Database

6.3.2 SELECT Queries

6.3.3 WHERE Clause

6.3.4 ORDER BY Clause

6.3.5 Merging Data from Multiple Tables: INNER JOIN

6.3.6 INSERT INTO Statement

16.3.7 UPDATE Statement

16.3.8 DELETE FROM Statement

6.3 NoSQL and NewSQL Big-Data Databases: A Brief Tour

6.3.1 NoSQL Key–Value Databases

6.3.2 NoSQL Document Databases

6.3.3 NoSQL Columnar Databases

6.3.4 NoSQL Graph Databases

6.3.5 NewSQL Databases

6.4 Case Study: A MongoDB JSON Document Database

6.4.1 Creating the MongoDB Atlas Cluster

6.4.2 Streaming Tweets into MongoDB

6.5 Hadoop

6.5.1 Hadoop Overview

6.6.2 Summarizing Word Lengths in *Romeo and Juliet* via MapReduce

6.5.3 Creating an Apache Hadoop Cluster in Microsoft Azure HDInsight

6.5.4 Hadoop Streaming

6.5.5 Implementing the Mapper

6.5.6 Implementing the Reducer

6.5.7 Preparing to Run the MapReduce Example

6.5.8 Running the MapReduce Job

6.6 Spark

6.6.1 Spark Overview

6.6.2 Docker and the Jupyter Docker Stacks

6.6.3 Word Count with Spark

6.6.4 Spark Word Count on Microsoft Azure

6.7 Spark Streaming: Counting Twitter Hashtags Using the `pyspark-notebook` Docker
tack

6.7.1 Streaming Tweets to a Socket

6.7.2 Summarizing Tweet Hashtags; Introducing Spark SQL

6.8 Internet of Things and Dashboards

6.8.1 Publish and Subscribe

6.8.2 Visualizing a PubNub Sample Live Stream with a Freeboard Dashboard

6.8.3 Simulating an Internet-Connected Thermostat in Python

6.8.4 Creating the Dashboard with Freeboard.io

6.8.5 Creating a Python PubNub Subscriber

6.9 Wrap-Up

16.1 INTRODUCTION

In [Section 1.7](#), we introduced big data. In this capstone chapter, we discuss popular hardware and software infrastructure for working with big data, and we develop complete applications on several desktop and cloud-based big-data platforms.

Databases

Databases are critical big-data infrastructure for storing and manipulating the massive amounts of data we’re creating. They’re also critical for securely and confidentially maintaining that data, especially in the context of ever-stricter privacy laws such as **HIPAA (Health Insurance Portability and Accountability Act)** in the United States and **GDPR (General Data Protection Regulation)** for the European Union.

First, we’ll present **relational databases**, which store **structured data** in tables with a fixed-size number of columns per row. You’ll manipulate relational databases via **Structured Query Language (SQL)**.

Most data produced today is **unstructured data**, like the content of Facebook posts and Twitter tweets, or **semi-structured data** like JSON and XML documents. Twitter processes each tweet’s contents into a semi-structured JSON document with lots of *metadata*, as you saw in the “Data Mining Twitter” chapter. Relational databases are not geared to the unstructured and semi-structured data in big-data applications. So, as big data evolved, new kinds of databases were created to handle such data efficiently. We’ll discuss the four major types of these **NoSQL databases**—key–value, document, columnar and graph databases. Also, we’ll overview **NewSQL databases**, which blend the benefits of relational and NoSQL databases. Many NoSQL and NewSQL vendors make it easy to get started with their products through free tiers and free trials, and typically in cloud-based environments that require minimal installation and setup. This makes it practical for you to gain big-data experience before “diving in.”

Apache Hadoop

Much of today’s data is so large that it cannot fit on one system. As big data grew, we needed

istributed data storage and parallel processing capabilities to process the data more efficiently. This led to complex technologies like Apache Hadoop for distributed data processing with massive parallelism among clusters of computers where the intricate details are handled for you automatically and correctly. We'll discuss Hadoop, its architecture and how it's used in big-data applications. We'll guide you through configuring a multi-node Hadoop cluster using the Microsoft Azure HDInsight cloud service, then use it to execute a Hadoop MapReduce job that you'll implement in Python. Though HDInsight is not free, Microsoft gives you a generous new-account credit that should enable you to run the chapter's code examples without incurring additional charges.

Apache Spark

As big-data processing needs grow, the information-technology community is continually looking for ways to increase performance. Hadoop executes tasks by breaking them into pieces that do lots of disk I/O across many computers. Spark was developed as a way to perform certain big-data tasks *in memory* for better performance.

We'll discuss Apache Spark, its architecture and how it's used in high-performance, real-time big-data applications. You'll implement a Spark application using functional-style filter/map/reduce programming capabilities. First, you'll build this example using a Jupyter Docker stack that runs locally on your desktop computer, then you'll implement it using a cloud-based Microsoft Azure HDInsight multi-node Spark cluster.

We'll introduce Spark streaming for processing streaming data in mini-batches. Spark streaming gathers data for a short time interval you specify, then gives you that batch of data to process. You'll implement a Spark streaming application that processes tweets. In that example, you'll use Spark SQL to query data stored in a Spark `DataFrame` which, unlike pandas `DataFrames`, may contain data distributed over many computers in a cluster.

Internet of Things

We'll conclude with an introduction to the Internet of Things (IoT)—billions of devices that are continuously producing data worldwide. We'll introduce the *publish/subscribe model* that IoT and other types of applications use to connect data users with data providers. First, without writing any code, you'll build a web-based dashboard using Freeboard.io and a sample live stream from the PubNub messaging service. Next, you'll simulate an Internet-connected thermostat which publishes messages to the free Dweet.io messaging service using the Python module Dweepy, then create a dashboard visualization of the data with Freeboard.io. Finally, you'll build a Python client that *subscribes* to a sample live stream from the PubNub service and dynamically visualizes the stream with Seaborn and a Matplotlib `FuncAnimation`.

Experience Cloud and Desktop Big-Data Software

Cloud vendors focus on **service-oriented architecture (SOA)** technology in which they provide “as-a-Service” capabilities that applications connect to and use in the cloud. Common services provided by cloud vendors include: ¹

¹ For more as-a-Service acronyms, see
https://en.wikipedia.org/wiki/Cloud_computing and
https://en.wikipedia.org/wiki/As_a_service.

“As-a-Service” acronyms (note that several are the same)

Big data as a Service (BDaaS)	Platform as a Service (PaaS)
Hadoop as a Service (HaaS)	Software as a Service (SaaS)
Hardware as a Service (HaaS)	Storage as a Service (SaaS)
Infrastructure as a Service (IaaS)	Spark as a Service (SaaS)

You’ll get hands-on experience in this chapter with several cloud-based tools. In this chapter’s examples, you’ll use the following platforms:

- A *free* MongoDB Atlas cloud-based cluster.
- A multi-node Hadoop cluster running on Microsoft’s Azure HDInsight cloud-based service—for this you’ll use the *credit* that comes with a new Azure account.
- A *free* single-node Spark “cluster” running on your desktop computer, using a Jupyter Docker-stack container.
- A multi-node Spark cluster, also running on Microsoft’s Azure HDInsight—for this you’ll continue using your Azure new-account *credit*.

There are many other options, including cloud-based services from Amazon Web Services, Google Cloud and IBM Watson, and the free *desktop* versions of the Hortonworks and Cloudera platforms (there also are cloud-based paid versions of these). You also could try a single-node Spark cluster running on the *free* cloud-based Databricks Community Edition. Spark’s creators founded Databricks.

Always check the latest terms and conditions of each service you use. Some require you to enable credit-card billing to use their clusters. *Caution:* Once you allocate Microsoft Azure HDInsight clusters (or other vendors’ clusters), they incur costs. When you complete the case studies using services such as Microsoft Azure, be sure to delete your cluster(s) and their other resources (like storage). This will help extend the life of your Azure new-account credit.

Installation and setups vary across platforms and over time. Always follow each vendor’s latest steps. If you have questions, the best sources for help are the vendor’s support capabilities and forums. Also, check sites such as `stackoverflow.com`—other people may have asked questions about similar problems and received answers from the developer community.

Algorithms and Data

Algorithms and data are the core of Python programming. The first few chapters of this book were mostly about algorithms. We introduced control statements and discussed algorithm development. Data was small—primarily individual integers, floats and strings. hapters 5–

emphasized *structuring* data into lists, tuples, dictionaries, sets, arrays and files.

Data's Meaning

But, what about the *meaning* of the data? Can we use the data to gain insights to better diagnose cancers? Save lives? Improve patients' quality of life? Reduce pollution? Conserve water? Increase crop yields? Reduce damage from devastating storms and fires? Develop better treatment regimens? Create jobs? Improve company profitability?

The data-science case studies of [chapters 11– 5](#) all focused on AI. In this chapter, we focus on the big-data infrastructure that supports AI solutions. As the data used with these technologies continues growing exponentially, we want to learn from that data and do so at blazing speed. We'll accomplish these goals with a combination of sophisticated algorithms, hardware, software and networking designs. We've presented various machine-learning technologies, seeing that there are indeed great insights to be mined from data. With more data, and especially with big data, machine learning can be even more effective.

Big-Data Sources

The following articles and sites provide links to hundreds of free big data sources:

ig-data sources

"Awesome-Public-Datasets," GitHub.com,
<https://github.com/caesar0301/awesome-public-datasets>.

"AWS Public Datasets," <https://aws.amazon.com/public-datasets/>.

"Big Data And AI: 30 Amazing (And Free) Public Data Sources For 2018," by
<https://www.forbes.com/sites/bernardmarr/2018/02/26/big-data-and-amazing-and-free-public-data-sources-for-2018/>.

"Datasets for Data Mining and Data Science,"
<http://www.kdnuggets.com/datasets/index.html>.

"Exploring Open Data Sets," <https://datascience.berkeley.edu/open-datasets/>.

"Free Big Data Sources," Datamics, <http://datamics.com/free-big-data-sources/>.

Hadoop Illuminated, chapter 16. Publicly Available Big Data Sets,
http://hadoopilluminated.com/hadoop_illuminated/Public_Bigdata_Sources/.

List of Public Data Sources Fit for Machine Learning,”

<https://blog.bigml.com/list-of-public-data-sources-fit-for-machine-learning/>.

“Open Data,” Wikipedia, https://en.wikipedia.org/wiki/Open_data.

“Open Data 500 Companies,” <http://www.opendata500.com/us/list/>.

“Other Interesting Resources/Big Data and Analytics Educational Resources Research,” B. Marr, http://computing.derby.ac.uk/bigdatares/?page_id=

“6 Amazing Sources of Practice Data Sets,”

<https://www.jigsawacademy.com/6-amazing-sources-of-practice-data->

“20 Big Data Repositories You Should Check Out,” M. Krivanek,

<http://www.datasciencecentral.com/profiles/blogs/20-free-big-data-sources-everyone-should-check-out>.

“70+ Websites to Get Large Data Repositories for Free,”

<http://bigdata-madesimple.com/70-websites-to-get-large-data-repositories-for-free/>.

“Ten Sources of Free Big Data on Internet,” A. Brown,

<https://www.linkedin.com/pulse/ten-sources-free-big-data-internet-rown>.

“Top 20 Open Data Sources,”

<https://www.linkedin.com/pulse/top-20-open-data-sources-zygimantaciikevicius>.

“We’re Setting Data, Code and APIs Free,” NASA, <https://open.nasa.gov/openata/>.

“Where Can I Find Large Datasets Open to the Public?” Quora,

<https://www.quora.com/Where-can-I-find-large-datasets-open-to-the>

6.2 RELATIONAL DATABASES AND STRUCTURED QUERY LANGUAGE (SQL)

Databases are crucial, especially for big data. In [chapter 9](#), we demonstrated sequential text-file processing, working with data from CSV files and working with JSON. Both are useful when *most or all* of a file's data is to be processed. On the other hand, in transaction processing we need to locate and, possibly, update an *individual* data item quickly.

A **database** is an integrated collection of data. A **database management system (DBMS)** provides mechanisms for storing and organizing data in a manner consistent with the database's format. Database management systems allow for convenient access and storage of data without concern for the internal representation of databases.

Relational database management systems (RDBMSs) store data in **tables** and define relationships among the tables. Structured Query Language (SQL) is used almost universally with relational database systems to manipulate data and perform **queries**, which request information that satisfies given criteria. ²

² The writing in this chapter assumes that SQL is pronounced as see-quel. Some prefer ess que el.

Popular *open-source* RDBMSs include SQLite, PostgreSQL, MariaDB and MySQL. These can be downloaded and used *freely* by anyone. All have support for Python. We'll use SQLite, which is bundled with Python. Some popular proprietary RDBMSs include Microsoft SQL Server, Oracle, Sybase and IBM Db2.

Tables, Rows and Columns

A relational database is a logical table-based representation of data that allows the data to be accessed without consideration of its physical structure. The following diagram shows a sample `Employee` table that might be used in a personnel system:

	Number	Name	Department	Salary	Location
Row {	23603	Jones	413	1100	New Jersey
	24568	Kerwin	413	2000	New Jersey
	34589	Larson	642	1800	Los Angeles
	35761	Myers	611	1400	Orlando
	47132	Neumann	413	9000	New Jersey
	78321	Stephens	611	8500	Orlando
	Primary key		Column		

The table's primary purpose is to store employees' attributes. Tables are composed of **rows**, each describing a single entity. Here, each row represents one employee. Rows are composed of **columns** containing individual attribute values. The table above has six rows. The `Number` column represents the **primary key**—a column (or group of columns) with a value that's *unique* for each row. This guarantees that each row can be identified by its primary key. Examples of primary keys are Social Security numbers, employee ID numbers and part numbers in an inventory system—values in each of these are guaranteed to be unique. In this case, the rows are listed in ascending order by primary key, but they could be listed in

descending order or no particular order at all.

Each column represents a different data attribute. Rows are unique (by primary key) within a table, but particular column values may be duplicated between rows. For example, three different rows in the `Employee` table's `Department` column contain number 413.

Selecting Data Subsets

Different database users are often interested in different data and different relationships among the data. Most users require only subsets of the rows and columns. Queries specify which subsets of the data to select from a table. You use Structured Query Language (SQL) to define queries. For example, you might select data from the `Employee` table to create a result that shows where each department is located, presenting the data sorted in increasing order by department number. This result is shown below. We'll discuss SQL shortly.

[click here to view code image](#)

Department	Location
413	New Jersey
611	Orlando
642	Los Angeles

SQLite

The code examples in the rest of [Section 16.2](#) use the open-source SQLite database management system that's included with Python, but most popular database systems have Python support. Each typically provides a module that adheres to Python's **Database Application Programming Interface (DB-API)**, which specifies common object and method names for manipulating any database.

16.2.1 A `books` Database

In this section, we'll present a `books` database containing information about several of our books. We'll set up the database in SQLite via the Python Standard Library's **`sqlite3` module**, using a script provided in the `ch16` example's folder's `sql` subfolder. Then, we'll introduce the database's tables. We'll use this database in an IPython session to introduce various database concepts, including operations that **create, read, update** and **delete** data—the so-called **CRUD** operations. As we introduce the tables, we'll use SQL and pandas `DataFrames` to show you each table's contents. Then, in the next several sections, we'll discuss additional SQL features.

Creating the `books` Database

In your Anaconda Command Prompt, Terminal or shell, change to the `ch16` examples folder's `sql` subfolder. The following **`sqlite3` command** creates a SQLite database named `books.db` and executes the `books.sql` SQL script, which defines how to create the database's tables and populates them with data:

```
sqlite3 books.db < books.sql
```

The notation `<` indicates that `books.sql` is input into the `sqlite3` command. When the

command completes, the database is ready for use. Begin a new IPython session.

Connecting to the Database in Python

To work with the database in Python, first call `sqlite3`'s **connect function** to connect to the database and obtain a **Connection** object:

[click here to view code image](#)

```
In [1]: import sqlite3

In [2]: connection = sqlite3.connect('books.db')
```

authors Table

The database has three tables—`authors`, `author_isbn` and `titles`. The `authors` table stores all the authors and has three columns:

- `id`—The author's unique ID number. This integer column is defined as **autoincremented**—for each row inserted in the table, SQLite increases the `id` value by 1 to ensure that each row has a unique value. This column is the table's primary key.
- `first`—The author's first name (a string).
- `last`—The author's last name (a string).

Viewing the authors Table's Contents

Let's use a SQL query and pandas to view the `authors` table's contents:

[click here to view code image](#)

```
In [3]: import pandas as pd

In [4]: pd.options.display.max_columns = 10

In [5]: pd.read_sql('SELECT * FROM authors', connection,
...:                 index_col=['id'])
...:
Out[5]:
   first  last
id
1    Paul Deitel
2   Harvey Deitel
3    Abbey Deitel
4     Dan  Quirk
5 Alexander  Wald
```

Pandas function **read_sql** executes a SQL query and returns a `DataFrame` containing the query's results. The function's arguments are:

- a string representing the SQL query to execute,
- the SQLite database's `Connection` object, and in this case

- an `index_col` keyword argument indicating which column should be used as the DataFrame's row indices (the author's `id` values in this case).

As you'll see momentarily, when `index_col` is not passed, index values starting from 0 appear to the left of the DataFrame's rows.

A SQL **SELECT** query gets rows and columns from one or more tables in a database. In the query:

```
SELECT * FROM authors
```

the **asterisk (*)** is a *wildcard* indicating that the query should get *all* the columns from the `authors` table. We'll discuss `SELECT` queries in more detail shortly.

titles Table

The `titles` table stores all the books and has four columns:

- `isbn`—The book's ISBN (a string) is this table's primary key. ISBN is an abbreviation for "International Standard Book Number," which is a numbering scheme that publishers use to give every book a unique identification number.
- `title`—The book's title (a string).
- `edition`—The book's edition number (an integer).
- `copyright`—The book's copyright year (a string).

Let's use SQL and pandas to view the `titles` table's contents:

[click here to view code image](#)

```
In [6]: pd.read_sql('SELECT * FROM titles', connection)
Out[6]:
```

	isbn	title	edition	copyright
0	0135404673	Intro to Python for CS and DS	1	2020
1	0132151006	Internet & WWW How to Program	5	2012
2	0134743350	Java How to Program	11	2018
3	0133976890	C How to Program	8	2016
4	0133406954	Visual Basic 2012 How to Program	6	2014
5	0134601548	Visual C# How to Program	6	2017
6	0136151574	Visual C++ How to Program	2	2008
7	0134448235	C++ How to Program	10	2017
8	0134444302	Android How to Program	3	2017
9	0134289366	Android 6 for Programmers	3	2016

author_ISBN Table

The `author_ISBN` table uses the following columns to associate authors from the `authors` table with their books in the `titles` table:

- `id`—An author's `id` (an integer).

- `isbn`—The book's ISBN (a string).

The `id` column is a **foreign key**, which is a column in this table that matches a *primary-key* column in another table—in particular, the `authors` table's `id` column. The `isbn` column also is a foreign key—it matches the `titles` table's `isbn` primary-key column. A database might have many tables. A goal when designing a database is to *minimize data duplication* among the tables. To do this, each table represents a specific entity, and foreign keys help link the data in *multiple* tables. The primary keys and foreign keys are designated when you create the database tables (in our case, in the `books.sql` script).

Together the `id` and `isbn` columns in this table form a *composite primary key*. Every row in this table *uniquely* matches *one* author to *one* book's ISBN. This table contains many entries, so let's use SQL and pandas to view just the first five rows:

[lick here to view code image](#)

```
In [7]: df = pd.read_sql('SELECT * FROM author_ISBN', connection)

In [8]: df.head()
Out[8]:
```

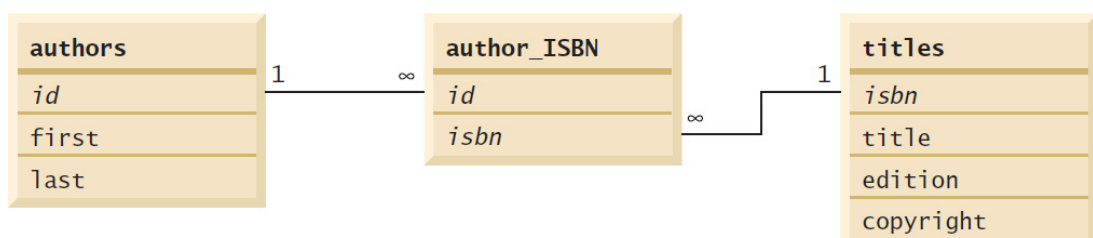
	id	isbn
0	1	0134289366
1	2	0134289366
2	5	0134289366
3	1	0135404673
4	2	0135404673

Every foreign-key value must appear as the primary-key value in a row of another table so the DBMS can ensure that the foreign-key value is valid. This is known as the **Rule of Referential Integrity**. For example, the DBMS ensures that the `id` value for a particular `author_ISBN` row is valid by checking that there is a row in the `authors` table with that `id` as the primary key.

Foreign keys also allow *related* data in *multiple* tables to be *selected* from those tables and combined—this is known as **joining** the data. There is a **one-to-many relationship** between a primary key and a corresponding foreign key—one author can write many books, and similarly one book can be written by many authors. So a foreign key can appear *many* times in its table but only *once* (as the primary key) in another table. For example, in the `books` database, the ISBN 0134289366 appears in several `author_ISBN` rows because this book has several authors, but it appears only once as a primary key in `titles`.

Entity-Relationship (ER) Diagram

The following **entity-relationship- (ER) diagram** for the `books` database shows the database's *tables* and the *relationships* among them:



he first compartment in each box contains the table’s name, and the remaining compartments contain the table’s columns. The names in italic are primary keys. *A table’s primary key uniquely identifies each row in the table.* Every row must have a primary-key value, and that value must be unique in the table. This is known as the **Rule of Entity Integrity**. Again, for the `author_ISBN` table, the primary key is the combination of both columns—this is known as a composite primary key.

The lines connecting the tables represent the *relationships* among the tables. Consider the line between `authors` and `author_ISBN`. On the `authors` end there’s a 1, and on the `author_ISBN` end there’s an infinity symbol (∞). This indicates a *one-to-many relationship*. For *each* author in the `authors` table, there can be an *arbitrary number* of ISBNs for books written by that author in the `author_ISBN` table—that is, an author can write *any* number of books, so an author’s `id` can appear in multiple rows of the `author_ISBN` table. The relationship line links the `id` column in the `authors` table (where `id` is the primary key) to the `id` column in the `author_ISBN` table (where `id` is a foreign key). The line between the tables links the primary key to the matching foreign key.

The line between the `titles` and `author_ISBN` tables illustrates a *one-to-many relationship*—one book can be written by many authors. The line links the primary key `isbn` in table `titles` to the corresponding foreign key in table `author_ISBN`. The relationships in the entity-relationship diagram illustrate that the sole purpose of the `author_ISBN` table is to provide a **many-to-many relationship** between the `authors` and `titles` tables—an author can write *many* books, and a book can have *many* authors.

SQL Keywords

The following subsections continue our SQL presentation in the context of our `books` database, demonstrating SQL queries and statements using the SQL keywords in the following table. Other SQL keywords are beyond this text’s scope:

SQL keyword	Description
SELECT	Retrieves data from one or more tables.
FROM	Tables involved in the query. Required in every SELECT.
WHERE	Criteria for selection that determine the rows to be retrieved, deleted or updated. Optional in a SQL statement.
GROUP BY	Criteria for grouping rows. Optional in a SELECT query.
ORDER	

BY	Criteria for ordering rows. Optional in a SELECT query.
INNER JOIN	Merge rows from multiple tables.
INSERT	Insert rows into a specified table.
UPDATE	Update rows in a specified table.
DELETE	Delete rows from a specified table.

16.2.2 SELECT Queries

The previous section used SELECT statements and the * wildcard character to get all the columns from a table. Typically, you need only a subset of the columns, especially in big data where you could have dozens, hundreds, thousands or more columns. To retrieve only specific columns, specify a comma-separated list of column names. For example, let's retrieve only the columns `first` and `last` from the `authors` table:

[lick here to view code image](#)

```
In [9]: pd.read_sql('SELECT first, last FROM authors', connection)
Out[9]:
```

	first	last
0	Paul	Deitel
1	Harvey	Deitel
2	Abbey	Deitel
3	Dan	Quirk
4	Alexander	Wald

16.2.3 WHERE Clause

You'll often select rows in a database that satisfy certain **selection criteria**, especially in big data where a database might contain millions or billions of rows. Only rows that satisfy the selection criteria (formally called **predicates**) are selected. SQL's **WHERE clause** specifies a query's selection criteria. Let's select the `title`, `edition` and `copyright` for all books with copyright years greater than 2016. String values in SQL queries are delimited by single (') quotes, as in '2016':

[lick here to view code image](#)

```
In [10]: pd.read_sql("""SELECT title, edition, copyright
...:                  FROM titles
...:                  WHERE copyright > '2016'""", connection)
Out[10]:
```

	title	edition	copyright
0	Intro to Python for CS and DS	1	2020
1	Java How to Program	11	2018
2	Visual C# How to Program	6	2017
3	C++ How to Program	10	2017
4	Android How to Program	3	2017

Pattern Matching: Zero or More Characters

The `WHERE` clause may contain the operators `<`, `>`, `<=`, `>=`, `=`, `<>` (not equal) and `LIKE`. Operator **LIKE** is used for **pattern matching**—searching for strings that match a given pattern. A pattern that contains the **percent (%)** wildcard character searches for strings that have *zero or more* characters at the percent character's position in the pattern. For example, let's locate all authors whose last name starts with the letter D:

[lick here to view code image](#)

```
In [11]: pd.read_sql("""SELECT id, first, last
...:                FROM authors
...:                WHERE last LIKE 'D%',
...:                connection, index_col=['id'])
...:
Out[11]:
   first  last
id
1   Paul  Deitel
2  Harvey  Deitel
3   Abbey  Deitel
```

Pattern Matching: Any Character

An **underscore (_)** in the pattern string indicates a single wildcard character at that position. Let's select the rows of all the authors whose last names start with any character, followed by the letter b, followed by any number of additional characters (specified by %):

[lick here to view code image](#)

```
In [12]: pd.read_sql("""SELECT id, first, last
...:                FROM authors
...:                WHERE first LIKE '_b%',
...:                connection, index_col=['id'])
...:
Out[12]:
   first  last
id
3   Abbey  Deitel
```

16.2.4 ORDER BY Clause

The **ORDER BY clause** sorts a query's results into ascending order (lowest to highest) or descending order (highest to lowest), specified with `ASC` and `DESC`, respectively. The default sorting order is ascending, so `ASC` is optional. Let's sort the titles in ascending order:

[lick here to view code image](#)

```
In [13]: pd.read_sql('SELECT title FROM titles ORDER BY title ASC',
```

```

...:         connection)
Out[13]:
           title
0      Android 6 for Programmers
1      Android How to Program
2          C How to Program
3      C++ How to Program
4  Internet & WWW How to Program
5  Intro to Python for CS and DS
6          Java How to Program
7  Visual Basic 2012 How to Program
8      Visual C# How to Program
9      Visual C++ How to Program

```

Sorting By Multiple Columns

To sort by multiple columns, specify a comma-separated list of column names after the `ORDER BY` keywords. Let's sort the authors' names by last name, then by first name for any authors who have the same last name:

[lick here to view code image](#)

```

In [14]: pd.read_sql("""SELECT  id, first, last
...:                  FROM authors
...:                  ORDER BY  last, first""",
...:                  connection, index_col=['id'])
...:
Out[14]:
           first  last
id
3      Abbey  Deitel
2      Harvey  Deitel
1          Paul  Deitel
4          Dan   Quirk
5  Alexander   Wald

```

The sorting order can vary by column. Let's sort the authors in descending order by last name and ascending order by first name for any authors who have the same last name:

[lick here to view code image](#)

```

In [15]: pd.read_sql("""SELECT  id, first, last
...:                  FROM authors
...:                  ORDER BY last  DESC, first ASC""",
...:                  connection, index_col=['id'])
...:
Out[15]:
           first  last
id
5  Alexander   Wald
4          Dan   Quirk
3      Abbey  Deitel
2      Harvey  Deitel
1          Paul  Deitel

```

Combining the WHERE and ORDER BY Clauses

The `WHERE` and `ORDER BY` clauses can be combined in one query. Let's get the `isbn`, `title`, `edition` and `copyright` of each book in the `titles` table that has a title ending with

'How to Program' and sort them in ascending order by title.

[lick here to view code image](#)

```
In [16]: pd.read_sql("""SELECT    isbn, title, edition, copyright
...:                        FROM titles
...:                        WHERE title    LIKE '%How to Program'
...:                        ORDER BY     title""", connection)
Out[16]:
```

	isbn	title	edition	copyright
0	0134444302	Android How to Program	3	2017
1	0133976890	C How to Program	8	2016
2	0134448235	C++ How to Program	10	2017
3	0132151006	Internet & WWW How to Program	5	2012
4	0134743350	Java How to Program	11	2018
5	0133406954	Visual Basic 2012 How to Program	6	2014
6	0134601548	Visual C# How to Program	6	2017
7	0136151574	Visual C++ How to Program	2	2008

16.2.5 Merging Data from Multiple Tables: INNER JOIN

Recall that the `books` database's `author_ISBN` table links authors to their corresponding titles. If we did not separate this information into individual tables, we'd need to include author information with each entry in the `titles` table. This would result in storing *duplicate* author information for authors who wrote multiple books.

You can merge data from multiple tables, referred to as joining the tables, with **INNER JOIN**. Let's produce a list of authors accompanied by the ISBNs for books written by each author—because there are many results for this query, we show just the head of the result:

[lick here to view code image](#)

```
In [17]: pd.read_sql("""SELECT    first, last, isbn
...:                        FROM authors
...:                        INNER JOIN    author_ISBN
...:                        ON    authors.id = author_ISBN.id
...:                        ORDER BY     last, first""", connection).head()
Out[17]:
```

	first	last	isbn
0	Abbey	Deitel	0132151006
1	Abbey	Deitel	0133406954
2	Harvey	Deitel	0134289366
3	Harvey	Deitel	0135404673
4	Harvey	Deitel	0132151006

The **INNER JOIN**'s **ON clause** uses a primary-key column in one table and a foreign-key column in the other to determine which rows to merge from each table. This query merges the `authors` table's `first` and `last` columns with the `author_ISBN` table's `isbn` column and sorts the results in ascending order by `last` then `first`.

Note the syntax `authors.id (table_name.column_name)` in the **ON** clause. This **qualified name** syntax is required if the columns have the same name in both tables. This syntax can be used in any SQL statement to distinguish columns in different tables that have the same name. In some systems, table names qualified with the database name can be used to perform cross-database queries. As always, the query can contain an **ORDER BY** clause.

16.2.6 INSERT INTO Statement

To this point, you’ve queried existing data. Sometimes you’ll execute SQL statements that *modify* the database. To do so, you’ll use a `sqlite3 Cursor` object, which you obtain by calling the `Connection`’s `cursor` method:

[lick here to view code image](#)

```
In [18]: cursor = connection.cursor()
```

The pandas method `read_sql` actually uses a `Cursor` behind the scenes to execute queries and access the rows of the results.

The **INSERT INTO** statement inserts a row into a table. Let’s insert a new author named Sue Red into the `authors` table by calling `Cursor` method `execute`, which executes its SQL argument and returns the `Cursor`:

[lick here to view code image](#)

```
In [19]: cursor = cursor.execute("""INSERT INTO authors (first, last)
...:                               VALUES ('Sue', 'Red')""")
...:
```

The SQL keywords **INSERT INTO** are followed by the table in which to insert the new row and a comma-separated list of column names in parentheses. The list of column names is followed by the SQL keyword **VALUES** and a comma-separated list of values in parentheses. The values provided must match the column names specified both in order and type.

We do not specify a value for the `id` column because it’s an autoincremented column in the `authors` table—this was specified in the script `books.sql` that created the table. For every new row, SQLite assigns a unique `id` value that is the next value in the autoincremented sequence (i.e., 1, 2, 3 and so on). In this case, Sue Red is assigned `id` number 6. To confirm this, let’s query the `authors` table’s contents:

[lick here to view code image](#)

```
In [20]: pd.read_sql('SELECT id, first, last FROM authors',
...:                  connection, index_col=['id'])
...:
Out[20]:
   id  first  last
1    1   Paul Deitel
2    2  Harvey Deitel
3    3   Abbey Deitel
4    4    Dan Quirk
5    5 Alexander Wald
6    6    Sue Red
```

Note Regarding Strings That Contain Single Quotes

SQL delimits strings with single quotes (`'`). A string containing a single quote, such as O’Malley, must have *two* single quotes in the position where the single quote appears (e.g.,

'O' 'Malley')). The first acts as an escape character for the second. Not escaping single-quote characters in a string that's part of a SQL statement is a SQL syntax error.

16.2.7 UPDATE Statement

An **UPDATE** statement modifies existing values. Let's assume that Sue Red's last name is incorrect in the database and update it to 'Black':

[lick here to view code image](#)

```
In [21]: cursor = cursor.execute("""UPDATE    authors SET last='Black'
...:                                     WHERE    last='Red' AND first='Sue'""")
```

The **UPDATE** keyword is followed by the table to update, the keyword **SET** and a comma-separated list of *column_name = value* pairs indicating the columns to change and their new values. The change will be applied to *every* row if you do not specify a **WHERE** clause. The **WHERE** clause in this query indicates that we should update only rows in which the last name is 'Red' and the first name is 'Sue'.

Of course, there could be multiple people with the same first and last name. To make a change to only one row, it's best to use the row's unique primary key in the **WHERE** clause. In this case, we could have specified:

```
WHERE id = 6
```

For statements that modify the database, the **Cursor** object's **rowcount** attribute contains an integer value representing the number of rows that were modified. If this value is 0, no changes were made. The following confirms that the **UPDATE** modified one row:

[lick here to view code image](#)

```
In [22]: cursor.rowcount
Out[22]: 1
```

We also can confirm the update by listing the **authors** table's contents:

[lick here to view code image](#)

```
In [23]: pd.read_sql('SELECT id,    first, last FROM authors',
...:                  connection, index_col=['id'])
...:
Out[23]:
   id  first  last
1    Paul  Deitel
2  Harvey  Deitel
3   Abbey  Deitel
4     Dan  Quirk
5 Alexander  Wald
6     Sue  Black
```

16.2.8 DELETE FROM Statement

A SQL **DELETE FROM** statement removes rows from a table. Let's remove Sue Black from the `authors` table using her author ID:

[click here to view code image](#)

```
In [24]: cursor = cursor.execute('DELETE FROM authors WHERE id=6')

In [25]: cursor.rowcount
Out[25]: 1
```

The optional `WHERE` clause determines which rows to delete. If `WHERE` is omitted, all the table's rows are deleted. Here's the `authors` table after the `DELETE` operation:

[click here to view code image](#)

```
In [26]: pd.read_sql('SELECT id, first, last FROM authors',
...:                  connection, index_col=['id'])
...:
Out[26]:
```

	id	first	last
1	1	Paul	Deitel
2	2	Harvey	Deitel
3	3	Abbey	Deitel
4	4	Dan	Quirk
5	5	Alexander	Wald

Closing the Database

When you no longer need access to the database, you should call the `Connection`'s `close` method to disconnect from the database:

```
connection.close()
```

SQL in Big Data

SQL's importance is growing in big data. Later in this chapter, we'll use Spark SQL to query data in a Spark `DataFrame` for which the data may be distributed over many computers in a Spark cluster. As you'll see, Spark SQL looks much like the SQL presented in this section.

16.3 NOSQL AND NEWSQL BIG-DATA DATABASES: A BRIEF TOUR

For decades, relational database management systems have been the standard in data processing. However, they require *structured data* that fits into neat rectangular tables. As the size of the data and the number of tables and relationships increases, relational databases become more difficult to manipulate efficiently. In today's big-data world, NoSQL and NewSQL databases have emerged to deal with the kinds of data storage and processing demands that traditional relational databases cannot meet. Big data requires massive databases, often spread across data centers worldwide in huge clusters of commodity computers. According to `statista.com`, there are currently over 8 million data centers worldwide. ³

³ <https://www.statista.com/statistics/500458/worldwide-datacenter-and-it-sites/>.

oSQL originally meant what its name implies. With the growing importance of SQL in big data—such as SQL on Hadoop and Spark SQL—NoSQL now is said to stand for “Not Only SQL.” NoSQL databases are meant for unstructured data, like photos, videos and the natural language found in e-mails, text messages and social-media posts, and semi-structured data like JSON and XML documents. Semi-structured data often wraps unstructured data with additional information called **metadata**. For example, YouTube videos are unstructured data, but YouTube also maintains metadata for each video, including who posted it, when it was posted, a title, a description, tags that help people discover the videos, privacy settings and more—all returned as JSON from the YouTube APIs. This metadata adds structure to the unstructured video data, making it semi-structured.

The next several subsections overview the four NoSQL database categories—key–value, document, columnar (also called column-based) and graph. In addition, we’ll overview NewSQL databases, which blend features of relational and NoSQL databases. In [Section 16.4](#), we’ll present a case study in which we store and manipulate a large number of JSON tweet objects in a NoSQL document database, then summarize the data in an interactive visualization displayed on a Folium map of the United States.

16.3.1 NoSQL Key–Value Databases

Like Python dictionaries, **key–value databases** ⁴ store key–value pairs, but they’re optimized for distributed systems and big-data processing. For reliability, they tend to replicate data in multiple cluster nodes. Some key–value databases, such as Redis, are implemented in memory for performance, and others store data on disk, such as HBase, which runs on top of Hadoop’s HDFS distributed file system. Other popular key–value databases include Amazon DynamoDB, Google Cloud Datastore and Couchbase. DynamoDB and Couchbase are **multi-model databases** that also support documents. HBase is also a column-oriented database.

⁴ https://en.wikipedia.org/wiki/Key-value_database.

16.3.2 NoSQL Document Databases

A **document database** ⁵ stores semi-structured data, such as JSON or XML documents. In document databases, you typically add indexes for specific attributes, so you can more efficiently locate and manipulate documents. For example, let’s assume you’re storing JSON documents produced by IoT devices and each document contains a type attribute. You might add an index for this attribute so you can filter documents based on their types. Without indexes, you can still perform that task, it will just be slower because you have to search each document in its entirety to find the attribute.

⁵ https://en.wikipedia.org/wiki/Document-oriented_database.

The most popular document database (and most popular overall NoSQL database ⁶) is **MongoDB**, whose name derives from a sequence of letters embedded in the word “humongous.” In an example, we’ll store a large number of tweets in MongoDB for processing. Recall that Twitter’s APIs return tweets in JSON format, so they can be stored directly in MongoDB. After obtaining the tweets we’ll summarize them in a pandas `Data-`

Frame and on a Folium map. Other popular document databases include Amazon DynamoDB (also a key–value database), Microsoft Azure Cosmos DB and Apache CouchDB.

⁶ <https://db-engines.com/en/ranking>.

16.3.3 NoSQL Columnar Databases

In a relational database, a common query operation is to get a specific column’s value for every row. Because data is organized into rows, a query that selects a specific column can perform poorly. The database system must get every matching row, locate the required column and discard the rest of the row’s information. A **columnar database** ^{7, 8}, also called a **column-oriented database**, is similar to a relational database, but it stores structured data in columns rather than rows. Because all of a column’s elements are stored together, selecting all the data for a given column is more efficient.

⁷ https://en.wikipedia.org/wiki/Columnar_database.

⁸ <https://www.predictiveanalyticstoday.com/top-wide-columnar-store-databases/>.

Consider our `authors` table in the `books` database:

[click here to view code image](#)

	first	last
id		
1	Paul	Deitel
2	Harvey	Deitel
3	Abbey	Deitel
4	Dan	Quirk
5	Alexander	Wald

In a relational database, all the data for a row is stored together. If we consider each row as a Python tuple, the rows would be represented as `(1, 'Paul', 'Deitel')`, `(2, 'Harvey', 'Deitel')`, etc. In a columnar database, all the values for a given column would be stored together, as in `(1, 2, 3, 4, 5)`, `('Paul', 'Harvey', 'Abbey', 'Dan', 'Alexander')` and `('Deitel', 'Deitel', 'Deitel', 'Quirk', 'Wald')`. The elements in each column are maintained in row order, so the value at a given index in each column belongs to the same row. Popular columnar databases include MariaDB ColumnStore and HBase.

16.3.4 NoSQL Graph Databases

A graph models relationships between objects. ⁹ The objects are called **nodes** (or **vertices**) and the relationships are called **edges**. Edges are *directional*. For example, an edge representing an airline flight points from the origin city to the destination city, but not the reverse. A **graph database** ⁰ stores nodes, edges and their attributes.

⁹ https://en.wikipedia.org/wiki/Graph_theory.

⁰ https://en.wikipedia.org/wiki/Graph_database.

If you use social networks, like Instagram, Snapchat, Twitter and Facebook, consider your

social graph, which consists of the people you know (nodes) and the relationships between them (edges). Every person has their own social graph, and these are interconnected. The famous “six degrees of separation” problem says that any two people in the world are connected to one another by following a maximum of six edges in the worldwide social graph.¹ Facebook’s algorithms use the social graphs of their billions of monthly active users² to determine which stories should appear in each user’s news feed. By looking at your interests, your friends, their interests and more, Facebook predicts the stories they believe are most relevant to you.³

¹ https://en.wikipedia.org/wiki/Six_degrees_of_separation.

² <https://zephoria.com/top-15-valuable-facebook-statistics/>.

³ <https://newsroom.fb.com/news/2018/05/inside-feed-news-feed-ranking/>.

Many companies use similar techniques to create recommendation engines. When you browse a product on Amazon, they use a graph of users and products to show you comparable products people browsed before making a purchase. When you browse movies on Netflix, they use a graph of users and movies they liked to suggest movies that might be of interest to you.

One of the most popular graph databases is Neo4j. Many real-world use-cases for graph databases are provided at:

<https://neo4j.com/graphgists/>

With most of the use-cases, sample graph diagrams produced by Neo4j are shown. These visualize the relationships between the graph nodes. Check out Neo4j’s free PDF book, *Graph Databases*.⁴

⁴ <https://neo4j.com/graph-databases-book-sx2>.

16.3.5 NewSQL Databases

Key advantages of relational databases include their security and transaction support. In particular, relational databases typically use **ACID (Atomicity, Consistency, Isolation, Durability)**⁵ transactions:

⁵ [https://en.wikipedia.org/wiki/ACID_\(computer_science\)](https://en.wikipedia.org/wiki/ACID_(computer_science)).

- *Atomicity* ensures that the database is modified only if *all* of a transaction’s steps are successful. If you go to an ATM to withdraw \$100, that money is not removed from your account unless you have enough money to cover the withdrawal *and* there is enough money in the ATM to satisfy your request.
- *Consistency* ensures that the database state is always valid. In the withdrawal example above, your new account balance after the transaction will reflect precisely what you withdrew from your account (and possibly ATM fees).
- *Isolation* ensures that concurrent transactions occur as if they were performed sequentially. For example, if two people share a joint bank account and both attempt to

withdraw money at the same time from two separate ATMs, one transaction must wait until the other completes.

- *Durability* ensures that changes to the database survive even hardware failures.

If you research benefits and disadvantages of NoSQL databases, you'll see that NoSQL databases generally do not provide ACID support. The types of applications that use NoSQL databases typically do not require the guarantees that ACID-compliant databases provide. Many NoSQL databases typically adhere to the **BASE (Basic Availability, Soft-state, Eventual consistency)** model, which focuses more on the database's availability. Whereas, ACID databases guarantee consistency when you write to the database, BASE databases provide consistency at some later point in time.

NewSQL databases blend the benefits of both relational and NoSQL databases for big-data processing tasks. Some popular NewSQL databases include VoltDB, MemSQL, Apache Ignite and Google Spanner.

16.4 CASE STUDY: A MONGODB JSON DOCUMENT DATABASE

MongoDB is a document database capable of storing and retrieving JSON documents. Twitter's APIs return tweets to you as JSON objects, which you can write directly into a MongoDB database. In this section, you'll:

- use Tweepy to stream tweets about the 100 U.S. senators and store them into a MongoDB database,
- use pandas to summarize the top 10 senators by tweet activity and
- display an interactive Folium map of the United States with one popup marker per state that shows the state name and both senators' names, their political parties and tweet counts.

You'll use a free cloud-based MongoDB Atlas cluster, which requires no installation and currently allows you to store up to 512MB of data. To store more, you can download the MongoDB Community Server from:

[lick here to view code image](#)

```
https://www.mongodb.com/download-center/community
```

and run it locally or you can sign up for MongoDB's *paid* Atlas service.

Installing the Python Libraries Required for Interacting with MongoDB

You'll use the **pymongo library** to interact with MongoDB databases from your Python code. You'll also need the `dnspython` library to connect to a MongoDB Atlas Cluster. To install these libraries, use the following commands:

[lick here to view code image](#)